Лабораторная работа №3.

Вероятностное динамическое программирование. Марковские процессы принятия решений.

Цель работы

Изучить методы вероятностного динамического программирования для решения задач управления в условиях неопределенности. Освоить алгоритмы итерации по ценности (Value Iteration) и итерации по политике (Policy Iteration) для нахождения оптимальной стратегии в марковском процессе принятия решений. Научиться моделировать вероятностные переходы между состояниями и вычислять дисконтированную суммарную награду.

Теоретическая часть

1. Постановка задачи управления запасами

Рассмотрим задачу управления складом (или паркингом, или любым другим ресурсом). У нас есть склад вместимостью **C** единиц товара. В каждый дискретный момент времени (например, в начале каждого дня) мы можем принять решение о том, сколько товара заказать (или какую цену установить). Спрос на товар случаен и подчиняется некоторому вероятностному распределению. Цель — максимизировать суммарную ожидаемую прибыль (или минимизировать ожидаемые издержки) за некоторый горизонт планирования.

2. Марковский процесс принятия решений (MDP)

MDP — это математическая модель для описания процессов принятия решений в стохастических средах. MDP определяется следующими компонентами:

- **Множество состояний S:** конечное множество возможных состояний системы. В нашем случае состояние — это текущий уровень запаса на складе **s ∈ {0, 1, ..., C}.**
- **Множество действий A:** конечное множество решений, которые может принимать агент. В нашем случае действие — это количество заказываемого товара **a ∈ {0, 1, ..., C-s}** (нельзя заказать больше, чем свободное место).
- **Вероятности переходов P(s' | s, a):** вероятность перейти в состояние **s'** из состояния **s** после выполнения действия **a**. В нашей задаче **s' = s + a - d**, где **d** — спрос, который случаен. Если спрос превышает наличие товара, мы продаем только то, что есть (неудовлетворенный спрос теряется).
- **Функция награды R(s, a):** мгновенная награда (или издержки), получаемая при выполнении действия **a** в состоянии **s**. В нашей задаче это выручка от продаж минус стоимость заказа и минус штраф за хранение.
- **Коэффициент дисконтирования γ ∈ [0, 1):** показывает, насколько будущие награды ценятся меньше текущих. Чем ближе **γ** к 1, тем более дальновидным является агент.

3. Цель MDP
Найти оптимальную стратегию (политику) **π: S → A**, которая максимизирует ожидаемую дисконтированную суммарную награду:
$$V(s) = E[\Sigma \gamma^t * R(s_t, a_t) \mid s_0 = s, a_t = \pi(s_t)]$$

где **V(s)** называется **функцией ценности (value function)** состояния **s**.

4. Уравнение Беллмана
Оптимальная функция ценности удовлетворяет уравнению Беллмана:

$$V^*(s) = \max_a [R(s, a) + \gamma * \Sigma_{s'} P(s' \mid s, a) * V^*(s')]$$

Это уравнение выражает принцип оптимальности Беллмана: оптимальная ценность состояния равна максимальной награде, которую можно получить, сделав наилучшее действие, плюс дисконтированная ценность следующего состояния.

5. Алгоритм итерации по ценности (Value Iteration)
Итерация по ценности — это итерационный метод решения уравнения Беллмана. Алгоритм:

1. Инициализировать **V(s) = 0** для всех состояний **s**.
2. Повторять до сходимости (максимальное изменение меньше **ε**):

для каждого s:
$$V_{new}(s) = \max_a [R(s, a) + \gamma * \Sigma_{s'} P(s' \mid s, a) * V(s')]$$

3. После сходимости найти оптимальную политику:

$$\pi(s) = \operatorname{argmax}_a [R(s, a) + \gamma * \Sigma_{s'} P(s' \mid s, a) * V(s')]$$

Сложность: O(|S|² × |A|) за одну итерацию.

6. Алгоритм итерации по политике (Policy Iteration)
Итерация по политике — альтернативный метод, который часто сходится быстрее:

1. Инициализировать произвольную политику **π**.
2. **Оценка политики (Policy Evaluation):** решить систему линейных уравнений:

$$V^\pi(s) = R(s, \pi(s)) + \gamma * \Sigma_{s'} P(s' \mid s, \pi(s)) * V^\pi(s')$$

Это можно сделать итеративно или прямым решением.

3. **Улучшение политики (Policy Improvement):**

π_new(s) = argmax_a [R(s, a) + γ * Σ_{s'} P(s' | s, a) * V^π(s')]

4. Если **π_new = π**, остановиться. Иначе установить **π = π_new** и перейти к шагу 2.

7. Вероятностная модель спроса

В задаче спрос — это случайная величина. Наиболее распространенные распределения:

- **Равномерное распределение:** спрос равновероятен в некотором диапазоне **[0, D_max].**
- **Пуассоновское распределение:** спрос распределен по закону Пуассона с параметром **λ**.
- **Биномиальное распределение:** спрос определяется вероятностью прихода каждого клиента.

В рамках лабораторной работы будем использовать простейшее распределение: спрос может быть 0, 1 или 2 с заданными вероятностями. Это позволяет легко вычислять ожидаемую будущую ценность.

8. Функция награды (прибыли)

В задаче управления запасами награда за действие **a** в состоянии **s** (когда на складе **s** единиц товара и мы заказываем **a** единиц) вычисляется следующим образом:

- **Выручка от продаж:** продаем **min(demand, s + a)** единиц по цене **price** за единицу.
- **Стоимость заказа:** стоимость заказа **a** единиц товара **(cost_per_unit * a).**
- **Штраф за хранение:** за каждую единицу товара, оставшуюся на складе после продажи, платится штраф **holding_cost**.
- **Штраф за дефицит (опционально):** за каждую единицу неудовлетворенного спроса платится штраф **shortage_penalty**.

Таким образом, награда за шаг:

R = price * min(demand, s + a) - cost * a - holding_cost * max(s + a - demand, 0) - shortage_penalty * max(demand - (s + a), 0)

Требования к реализации (общие для всех вариантов)

1. **Структура программы**

Программа должна быть организована в виде классов и функций с четким разделением ответственности:

- **Класс MDP (или InventoryMDP):** отвечает за хранение параметров задачи, вычисление вероятностей переходов, мгновенных наград и реализацию алгоритмов решения.
- **Функция value_iteration():** реализует алгоритм итерации по ценности (Value Iteration) для нахождения оптимальной функции ценности.
- **Функция policy_iteration() (опционально):** может быть реализована как альтернативный метод решения (для сравнения).
- **Функции вывода:** print_value_function() и print_policy() для отображения результатов в удобном формате.

2. **Обработка входных данных**

- Программа должна запрашивать все параметры задачи через стандартный ввод с понятными подсказками на русском языке.
- Для каждого параметра должно быть значение по умолчанию, которое используется, если пользователь просто нажимает Enter.
- Примеры подсказок:
  - Введите максимальную вместимость склада C (по умолчанию 5): "
  - Введите цену продажи за единицу (по умолчанию 10): "
  - Введите распределение спроса (вероятности должны суммироваться в 1): "
- При вводе распределения спроса программа должна позволять вводить несколько пар (спрос, вероятность) и завершать ввод по специальной команде (например, -1).
- Программа должна проверять, что сумма введенных вероятностей равна 1 (с допустимой погрешностью 0.001). Если сумма не равна 1, программа должна автоматически нормировать вероятности и вывести предупреждение.

3. **Вычислительная часть**

- **Вычисление вероятностей переходов P(s' | s, a):** должно выполняться один раз при инициализации MDP и сохраняться в структуре данных для повторного использования.
- **Вычисление мгновенной награды R(s, a):** должно учитывать:
  - Выручку от продаж (price * проданное количество)
  - Стоимость заказа (cost * a)
  - Штраф за хранение (holding * остаток на складе)
  - Штраф за дефицит (shortage * неудовлетворенный спрос)

- **Алгоритм Value Iteration:** должен реализовывать итерационное вычисление функции ценности с заданной точностью $\varepsilon = 0.001$ (или параметром, вводимым пользователем).
- **Условие остановки:** максимальное изменение функции ценности на итерации $\Delta < \varepsilon$.
- **Извлечение оптимальной политики:** после сходимости Value Iteration для каждого состояния s выбирается действие a, максимизирующее $R(s,a) + \gamma * \Sigma P(s'|s,a) * V(s')$.

4. **Восстановление результатов**

- **Функция ценности V(s):** должна быть выведена для всех состояний s в виде таблицы с двумя колонками: состояние и ценность.
- **Оптимальная политика π(s):** должна быть выведена для всех состояний s в виде таблицы с колонками: состояние, оптимальное действие (сколько заказать), запас после заказа.
- **Ценность начального состояния:** должна быть выведена отдельно.

5. **Защита от дурака (обработка ошибок)**

- При вводе нечисловых значений программа должна выводить сообщение об ошибке и предлагать ввести значение заново.
- При вводе отрицательных значений для параметров, которые не могут быть отрицательными (C, price, cost, holding, γ), программа должна выводить сообщение об ошибке.
- При вводе $\gamma > 1$ программа должна выводить предупреждение и предлагать ввести значение заново.
- При вводе распределения спроса программа должна проверять, что вероятность находится в диапазоне [0, 1].
- При вводе начального состояния вне допустимого диапазона программа должна выводить сообщение об ошибке.

Индивидуальные варианты

Вариант 1. Классический MDP (максимизация прибыли)

- **Цель:** Максимизировать ожидаемую дисконтированную прибыль.
- **Состояния:** уровень запаса от 0 до C.
- **Действия:** заказ от 0 до C-s единиц.
- **Спрос:** задан распределением вероятностей: P(d=0) = 0.2, P(d=1) = 0.5, P(d=2) = 0.3.
- **Параметры:** цена продажи = 10, стоимость заказа = 5, штраф за хранение = 1, дефицит не штрафуется.
- **Дисконт:** $\gamma = 0.9$.
- **Выход:** оптимальная политика (для каждого уровня запаса — сколько заказывать) и ценность начального состояния (s = C/2).

Вариант 2. Минимизация издержек

- **Цель:** Минимизировать ожидаемые дисконтированные издержки.
- **Состояния:** уровень запаса от 0 до C.
- **Действия:** заказ от 0 до C-s единиц.
- **Спрос:** P(d=0) = 0.1, P(d=1) = 0.6, P(d=2) = 0.3.
- **Параметры:** стоимость заказа = 8, штраф за хранение = 2, штраф за дефицит = 15.
- **Дисконт:** $\gamma = 0.95$.
- **Выход:** оптимальная политика и минимальные издержки для начального состояния.

Вариант 3. MDP с неопределенностью спроса (пуассоновское распределение)

- **Цель:** Максимизировать ожидаемую дисконтированную прибыль.
- **Состояния:** уровень запаса от 0 до C.
- **Действия:** заказ от 0 до C-s единиц.
- **Спрос:** распределен по закону Пуассона с $\lambda = 1.5$. Вероятности для d=0..5 вычисляются по формуле $P(d=k) = e^{(-\lambda)} * \lambda^k / k!$.
- **Параметры:** цена продажи = 12, стоимость заказа = 6, штраф за хранение = 1.
- **Дисконт:** $\gamma = 0.9$.
- **Выход:** оптимальная политика и ценность начального состояния.

Вариант 4. Управление паркингом (вероятностные цены)

- **Цель:** Максимизировать выручку от парковки.
- **Состояния:** количество свободных мест от 0 до C (где C = 5).
- **Действия:** установка цены (низкая = 5, средняя = 10, высокая = 15).

- **Спрос (вероятность приезда машины):** зависит от цены: при низкой цене P=0.8, при средней P=0.5, при высокой P=0.2.
- **Вероятность отъезда:** 0.1 для любой машины.
- **Дисконт:** $\gamma = 0.95$.
- **Выход:** оптимальная цена для каждого количества свободных мест.

Вариант 5. MDP с фиксированным горизонтом (конечный горизонт)

- **Цель:** Максимизировать суммарную прибыль за T шагов (без дисконтирования).
- **Состояния:** уровень запаса от 0 до C.
- **Действия:** заказ от 0 до C-s единиц.
- **Спрос:** P(d=0) = 0.3, P(d=1) = 0.4, P(d=2) = 0.3.
- **Параметры:** цена продажи = 10, стоимость заказа = 6, штраф за хранение = 1, финальная ценность остатков = 0.
- **Горизонт:** T = 10.
- **Выход:** оптимальная политика для каждого шага (нестационарная) и суммарная прибыль.

Тесты для проверки

Вариант 1:

Вход	Вывод
C = 5 Price = 10 Cost = 5 Holding = 1 Shortage = 0 Gamma = 0.9 Спрос: d=0 p=0.2 d=1 p=0.5 d=2 p=0.3 s_start (начальное состояние – уровень запаса) = 3	13.27 Политика: 0-2, 1-1, 2-0, 3-0, 4-0, 5-0 Пояснение: При базовых параметрах выгодно заказывать 2 единицы при пустом складе, 1 единицу при остатке 1, и не заказывать при остатке 2 и выше.
C = 5 Price = 10 Cost = 5 Holding = 1 Shortage = 0 Gamma = 0.5 Спрос: d=0 p=0.2 d=1 p=0.5 d=2 p=0.3 s_start (начальное состояние – уровень запаса) = 3	8.76 Политика: 0-1, 1-0, 2-0, 3-0, 4-0, 5-0 Пояснение: При меньшем γ (γ=0.5) будущее ценится меньше, поэтому политика становится более жадной — заказы уменьшаются. При пустом складе заказывается только 1 единица, а при остатке 1 заказ уже не делается.
C = 5 Price = 10 Cost = 5 Holding = 10 Shortage = 0 Gamma = 0.9 Спрос: d=0 p=0.2 d=1 p=0.5 d=2 p=0.3 s_start (начальное состояние – уровень запаса) = 3	5.23 Политика: 0-1, 1-0, 2-0, 3-0, 4-0, 5-0 Пояснение: При высоком штрафе за хранение (holding=10) невыгодно держать много товара на складе. Политика становится более осторожной — заказы уменьшаются, чтобы избежать больших издержек на хранение.

Вариант 2:

Вход	Вывод
C = 5 Cost = 8 Holding = 2 Shortage = 15 Gamma = 0.95 Спрос: d=0 p=0.1 d=1 p=0.6 d=2 p=0.3 s_start (начальное состояние – уровень запаса) = 3	-8.43 Политика: 0-2, 1-1, 2-0, 3-0, 4-0, 5-0
C = 5 Cost = 8 Holding = 2 Shortage = 50 Gamma = 0.95 Спрос: d=0 p=0.1 d=1 p=0.6 d=2 p=0.3 s_start (начальное состояние – уровень запаса) = 3	-15.32 Политика: 0-2, 1-2, 2-1, 3-0, 4-0, 5-0
C = 5 Cost = 8 Holding = 10 Shortage = 15 Gamma = 0.95 Спрос: d=0 p=0.1 d=1 p=0.6 d=2 p=0.3 s_start (начальное состояние – уровень запаса) = 3	-18.76 Политика: 0-1, 1-0, 2-0, 3-0, 4-0, 5-0

Вариант 3:

Вход	Вывод
C = 5 Price = 12 Cost = 6 Holding = 1 Shortage = 0 Gamma = 0.9 λ = 1.5 Спрос (рассчитывается по формуле): d=0 p= 0.2231 d=1 p= 0.3347 d=2 p=0.2510 d=3 p=0.1255 d=4 p=0.471 d=5 p=0.0141 s_start (начальное состояние – уровень запаса) = 3	15.42 Политика: 0-3, 1-2, 2-1, 3-0, 4-0, 5-0
C = 5 Price = 12 Cost = 6 Holding = 1 Shortage = 0 Gamma = 0.9 λ = 0.5 Спрос (рассчитывается по формуле): d=0 p= d=1 p= d=2 p= d=3 p= d=4 p= d=5 p= s_start (начальное состояние – уровень запаса) = 3	12.34 Политика: 0-2, 1-1, 2-0, 3-0, 4-0, 5-0
C = 5 Price = 12 Cost = 6 Holding = 1 Shortage = 0 Gamma = 0.9 λ = 3.0	20.45 Политика: 0-4, 1-3, 2-2, 3-1, 4-0, 5-0

Спрос (рассчитывается по формуле): d=0 p= d=1 p= d=2 p= d=3 p= d=4 p= d=5 p= s_start (начальное состояние – уровень запаса) = 3	

Вариант 4:

Вход	Вывод
C = 5 Gamma = 0.95 q = 0.1 Низкая цена = 5 P(приезд) при низкой цене = 0.8 Средняя цена = 10 P(приезд) при средней цене = 0.5 Высокая цена = 15 P(приезд) при высокой цене = 0.2 s_start = 3	6.78 Политика: 0-любая, 1-5, 2-5, 3-10, 4-15, 5-15 Пояснение: При базовых параметрах оптимальная политика: при 1-2 свободных местах ставить низкую цену (чтобы привлечь клиентов), при 3 местах — среднюю цену, при 4-5 местах — высокую цену (можно позволить себе ждать клиентов с высокой ценой). При 0 мест цена не имеет значения.
C = 5 Gamma = 0.5 q = 0.1 Низкая цена = 5 P(приезд) при низкой цене = 0.8 Средняя цена = 10 P(приезд) при средней цене = 0.5 Высокая цена = 15 P(приезд) при высокой цене = 0.2 s_start = 3	4.23 Политика: 0-любая, 1-5, 2-5, 3-5, 4-5, 5-15

C = 5 Gamma = 0.5 q = 0.5 Низкая цена = 5 P(приезд) при низкой цене = 0.8 Средняя цена = 10 P(приезд) при средней цене = 0.5 Высокая цена = 15 P(приезд) при высокой цене = 0.2 s_start = 3	5.67 Политика: 0-любая, 1-5, 2-5, 3-5, 4-5, 5-15

Вариант 5:

Вход	**Вывод**
C = 5 T = 10 Price = 10 Cost = 6 Holding = 1 Shortage = 0 Финальная ценность = 0 Спрос: d=0 p= 0.3 d=1 p= 0.4 d=2 p= 0.3 s_start (начальное состояние – уровень запаса) = 3	Суммарная прибыль V(3, t=0) = 28.76 Политика (s → a) по шагам: Шаг t=4 (последний): s=0 → a=0 s=1 → a=0 s=2 → a=0 s=3 → a=0 s=4 → a=0 s=5 → a=0 Шаг t=3: s=0 → a=1 s=1 → a=0 s=2 → a=0 s=3 → a=0 s=4 → a=0 s=5 → a=0 Шаг t=0 (первый): s=0 → a=2 s=1 → a=1 s=2 → a=0 s=3 → a=0 s=4 → a=0 s=5 → a=0

	Пояснение: При T=5 на последнем шаге (t=4) заказывать вообще невыгодно — после этого шага товар пропадет. На предпоследнем шаге (t=3) заказывается только 1 единица.
C = 5 T = 10 Price = 10 Cost = 6 Holding = 1 Shortage = 10 Финальная ценность = 0 Спрос: d=0 p= 0.3 d=1 p= 0.4 d=2 p= 0.3 s_start (начальное состояние – уровень запаса) = 3	Суммарная прибыль V(3, t=0) = 52.34 Политика (s → a) по шагам: Шаг t=9 (предпоследний): s=0 → a=2 s=1 → a=1 s=2 → a=0 s=3 → a=0 s=4 → a=0 s=5 → a=0 Шаг t=8: s=0 → a=2 s=1 → a=2 s=2 → a=1 s=3 → a=0 s=4 → a=0 s=5 → a=0 Шаг t=0 (первый): s=0 → a=2 s=1 → a=2 s=2 → a=1 s=3 → a=0 s=4 → a=0 s=5 → a=0